



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Shingamu Sai Ajay
24 April 2023



Outline



Executive
Summary



Introduction



Methodology



Results



Conclusion



Appendix

Executive Summary



Summary of methodologies

Data Collection

Data Wrangling

Web Scraping

EDA with SQL

EDA with Data Visualisation

Interactive Map using Folium

Interactive Dashboard using Plotly Dash

Predictive Analysis (Classification)



Summary of all results

Exploratory Data Analysis(EDA) results

Geospatial Visualisation

Interactive Dashboard

Predictive Analysis results

Introduction

Project background and context

SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars which is much cheaper than other providers that cost upward of 165 million dollars each. These cost savings are due to SpaceX's ability to reuse the first stage of their Falcon 9.

If we can determine whether the first stage of the Falcon 9 will land successfully, we will be able to better estimate the cost of a launch. This information can be used by other companies in order to decide whether to bid against SpaceX for a launch.

Thus the goal of this project is to predict if the first stage of the Falcon 9 will land successfully.

Problems you want to find answers

Is there a relationship between the launch variables and success of the first stage landing



Section 1

Methodology

Methodology

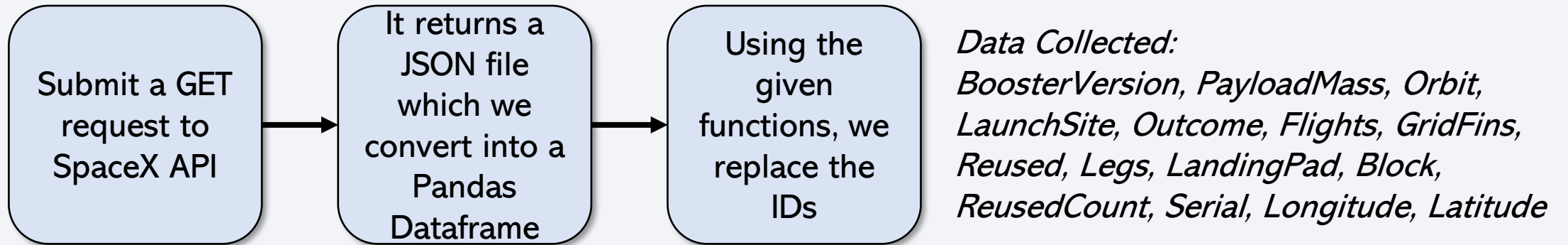
Executive Summary

- Data collection methodology:
 - Made GET requests to SpaceX API and Web Scraping from Wikipedia
- Perform data wrangling
 - The outcomes of landings were replaced with Training labels of 0/1 for failure/success.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Used Scikit-Learn to find best hyperparameters for SVM, Classification Trees, Logistic Regression and KNN and evaluated them based on their accuracy.

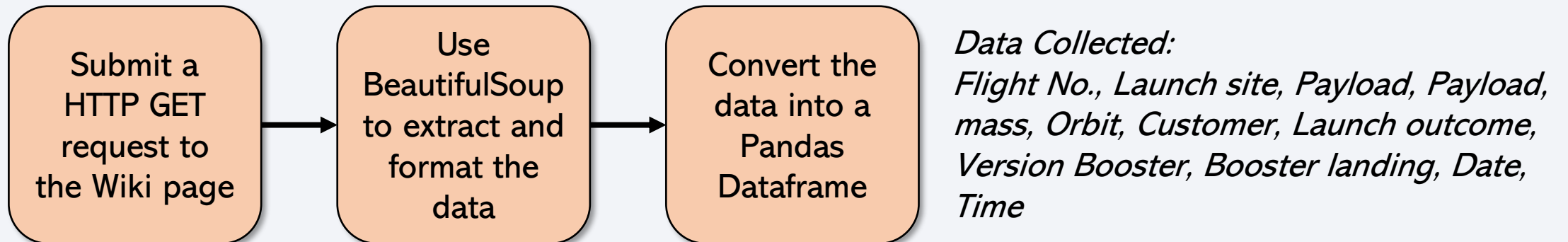
Data Collection

The data collection comprised on calling on SpaceX API and Web Scraping launch information from Wikipedia.

SpaceX API:



Web Scraping from Wikipedia:



Data Collection – SpaceX API

1. Make a GET request to SpaceX API
2. Decode response into JSON then convert to Pandas DataFrame
3. Clean the Data
 - Create lists to store data
 - Use custom functions to extract data
 - Create a dictionary using the data labels as columns
4. Create a new Pandas Dataframe using the dictionary
5. Filter the dataframe to only include Falcon 9 launches
6. Reset the FlightNumber column and replace null values in PayloadMass with its mean.

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
response = requests.get(spacex_url)
```

```
# Use json_normalize meethod to convert the json result into a dataframe  
data = pd.json_normalize(response.json())
```

```
#Global variables  
BoosterVersion = []  
PayloadMass = []  
Orbit = []  
LaunchSite = []  
Outcome = []  
Flights = []  
GridFins = []  
Reused = []  
Legs = []  
LandingPad = []  
Block = []  
ReusedCount = []  
Serial = []  
Longitude = []  
Latitude = []
```

```
# Call getBoosterVersion  
getBoosterVersion(data)
```

```
# Call getLaunchSite  
getLaunchSite(data)
```

```
# Call getPayloadData  
getPayloadData(data)
```

```
# Call getCoreData  
getCoreData(data)
```

```
# Create a data from launch_dict  
data = pd.DataFrame(launch_dict)
```

```
launch_dict = {'FlightNumber': list(data['flight_number']),  
'Date': list(data['date']),  
'BoosterVersion':BoosterVersion,  
'PayloadMass':PayloadMass,  
'Orbit':Orbit,  
'LaunchSite':LaunchSite,  
'Outcome':Outcome,  
'Flights':Flights,  
'GridFins':GridFins,  
'Reused':Reused,  
'Legs':Legs,  
'LandingPad':LandingPad,  
'Block':Block,  
'ReusedCount':ReusedCount,  
'Serial':Serial,  
'Longitude': Longitude,  
'Latitude': Latitude}
```

```
# Hint data['BoosterVersion']!='Falcon 1'  
data_falcon9 = data[data['BoosterVersion']=='Falcon 9']
```

```
data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))  
data_falcon9
```


Data Collection - Scraping

1. Make a HTTP GET request to Falcon9 Launch Wiki page
2. Create a BeautifulSoup object from the response
3. Find all the tables in the object
4. Extract the column names from the tables
5. Using the column names as keys, create a dictionary
6. Use custom functions to fill up the dictionary with launch records (View Appendix)
7. Create a Pandas Dataframe using the dictionary that can be exported

```
# use requests.get() method with the provided static_url
# assign the response to a object
page = requests.get(static_url)
```

```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(page.text, 'html.parser')
```

```
# Use the find_all function in the BeautifulSoup object, with element type `table`
# Assign the result to a list called `html_tables`
html_tables = (soup.find_all('table'))
```

```
# Apply find_all() function with `th` element on first_launch_table
column_names = []
temp = soup.find_all('th')
for x in range(len(temp)):
    try:
        name = extract_column_from_header(temp[x])
        if (name is not None and len(name) > 0):
            column_names.append(name)
    except:
        pass
```

```
launch_dict= dict.fromkeys(column_names)
```

```
# Remove an irrelevant column
del launch_dict['Date and time ( )']
```

```
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
launch_dict['Version Booster']=[[]]
launch_dict['Booster landing']=[[]]
launch_dict['Date']=[[]]
launch_dict['Time']=[[]]
```

```
headings = []
for key,values in dict(launch_dict).items():
    if key not in headings:
        headings.append(key)
    if values is None:
        del launch_dict[key]

def pad_dict_list(dict_list, padel):
    lmax = 0
    for lname in dict_list.keys():
        lmax = max(lmax, len(dict_list[lname]))
    for lname in dict_list.keys():
        ll = len(dict_list[lname])
        if ll < lmax:
            dict_list[lname] += [padel] * (lmax - ll)
    return dict_list

pad_dict_list(launch_dict,0)

df = pd.DataFrame.from_dict(launch_dict)
df
```

Data Wrangling

In this step, we will perform some Exploratory Data Analysis (EDA) to find some patterns in the data and determine what would be the label for training supervised models.

1. Number of launches on each site
2. Number and occurrence of each orbit
3. Number and occurrence of mission outcome per orbit type

```
# Apply value_counts() on column LaunchSite  
df['LaunchSite'].value_counts()
```

CCAFS SLC 40	55
KSC LC 39A	22
VAFB SLC 4E	13

1

```
# Apply value_counts on Orbit column  
df['Orbit'].value_counts()
```

GTO	27
ISS	21
VLEO	14
PO	9
LEO	7
SSO	5
MEO	3
ES-L1	1
HEO	1
SO	1
GEO	1

2

```
# landing_outcomes = values on Outcome column  
landing_outcomes = df['Outcome'].value_counts()  
landing_outcomes
```

True ASDS	41
None None	19
True RTLS	14
False ASDS	6
True Ocean	5
False Ocean	2
None ASDS	2
False RTLS	1

3

Data Wrangling

Different landing outcomes:

- **True Ocean** → mission outcome was **successfully** landed to a specific region of the ocean
- **False Ocean** → mission outcome was **unsuccessfully** landed to a specific region of the ocean
- **True RTLS** → mission outcome was **successfully** landed to a ground pad
- **False RTLS** → mission outcome was **unsuccessfully** landed to a ground pad
- **True ASDS** → mission outcome was **successfully** landed to a drone ship
- **False ASDS** → mission outcome was **unsuccessfully** landed to a drone ship
- **None ASDS** and **None None** → **failure** to land

Now we will convert these outcomes into Training Labels where 1 means successful and 0 means unsuccessful landing.

1. Create a set of outcomes where the second stage did not land successfully
2. Create a list that will contain the numerical outcomes
3. Assign the list to a new column in the dataframe called 'Class'

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])  
bad_outcomes
```

```
{'False ASDS', 'False Ocean', 'False RTLS', 'None ASDS', 'None None'}
```

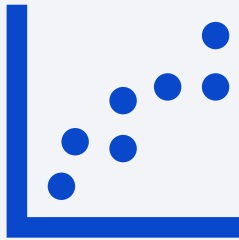
```
# landing_class = 0 if bad_outcome  
# landing_class = 1 otherwise  
landing_class = []  
for x in df['Outcome']:  
    if x in bad_outcomes:  
        landing_class.append(0)  
    else:  
        landing_class.append(1)
```

```
df['Class']=landing_class
```

EDA with Data Visualization

Scatter Chart

Used to visualise the relationship between 2 numerical variables

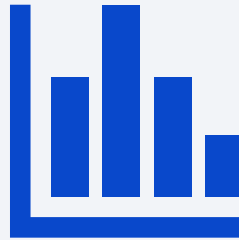


Relationships Observed:

- FlightNumber and LaunchSite
- PayloadMass and LaunchSite
- FlightNumber and Orbit type
- PayloadMass and Orbit type

Bar Chart

Used to visualise the relationship between a numerical and categorical variable

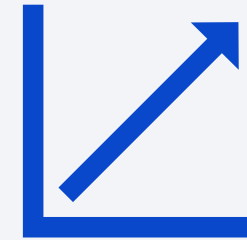


Relationships Observed:

- Orbit type and Success Rate

Line Chart

Used to visualise the relationship between a numerical and categorical variable



Relationships Observed:

- Orbit type and Success Rate

EDA with SQL

These are the SQL queries executed to learn more about the dataset:

- Display the names of the unique launch sites in the space mission
- Display 5 records where launch sites begin with the string 'CCA'
- Display the total payload mass carried by boosters launched by NASA (CRS)
- Display average payload mass carried by booster version F9 v1.1
- List the date when the first successful landing outcome in ground pad was achieved
- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
- List the total number of successful and failure mission outcomes
- List the names of the booster_versions which have carried the maximum payload mass. Use a subquery
- List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.
- Rank the count of successful landing_outcomes between the date 04-06-2010 and 20-03-2017 in descending order.

Build an Interactive Map with Folium

Map objects and their purposes:

1. Markers were placed on launch site locations on the map
 - This answers the following questions:
 - Are all launch sites in proximity to the Equator line?
 - Are all launch sites in very close proximity to the coast?
2. Markers show the success/failed launches for each site on the map
 - Since the launches only take place at 4 sites, the markers for each launch are clustered and assigned a colour. If a launch was successful (class=1), then we use a green marker and if a launch was failed, we use a red marker (class=0)
3. Lines were added between launch sites and its proximities
 - These lines indicate the distance between launch sites and its proximities.
 - This answers the following questions:
 - Are launch sites in close proximity to railways, highways and coastline? (Yes)
 - Do launch sites keep certain distance away from cities? (Yes)

Build a Dashboard with Plotly Dash

The interactive dashboard has a pie chart and scatter plot.

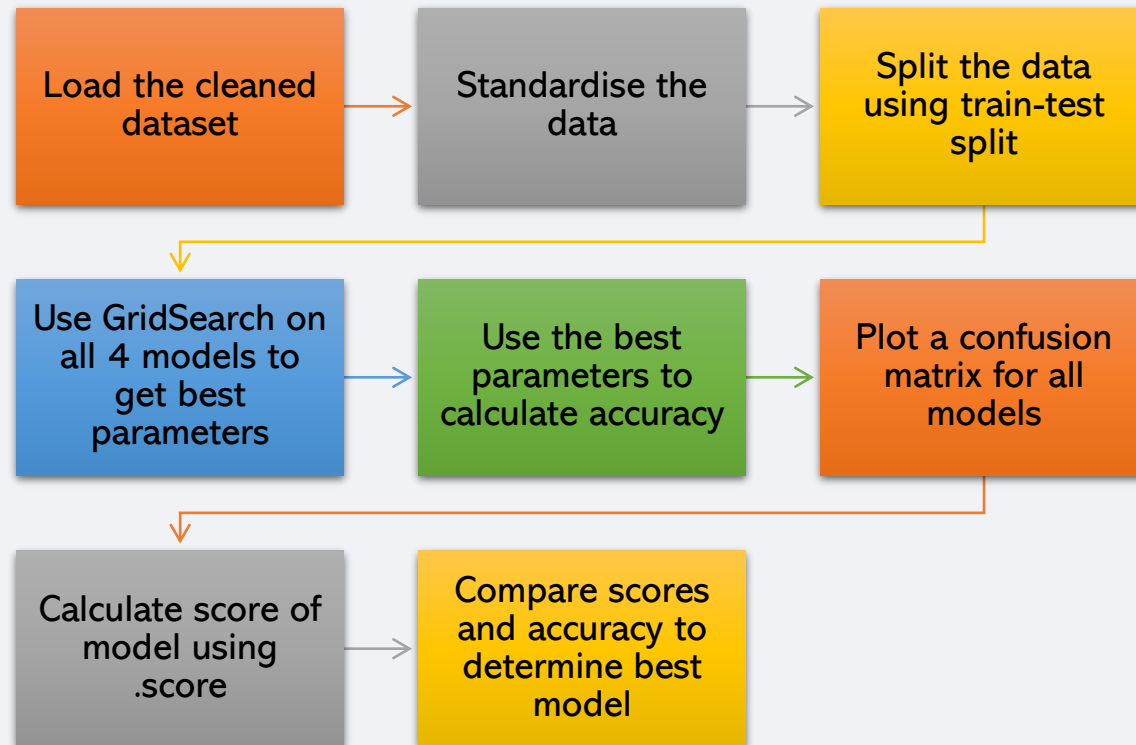
Pie Chart:

- It shows the total successful launches at each launch site
- The launch site with the largest slice of the pie chart has the most successful launches.
- The pie chart also has a dropdown feature that allows users to the success and failure rate of each individual site.

Scatter Plot:

- It shows the correlation between PayloadMass and Success for all sites.
- The dropdown can also narrow down the points to just the selected launch site
- The rangeslider can also filter the points further to a particular PayloadMass range.
- The points are also coloured based on the Booster_version used in the launch.

Predictive Analysis (Classification)

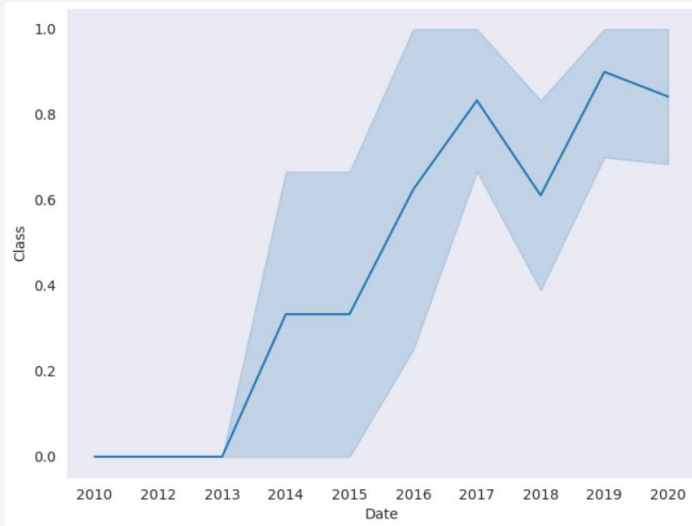


Models used:

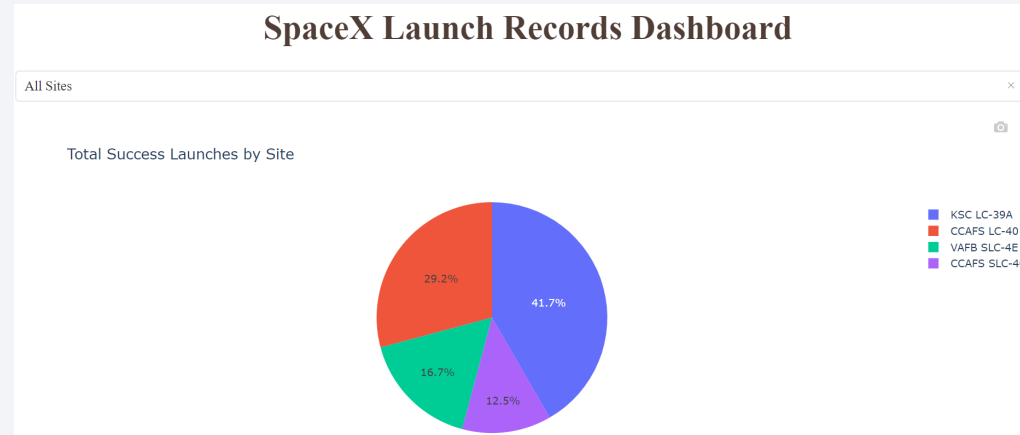
1. Logistic Regression
2. Support Vector Machine(SVM)
3. Decision Tree Classification
4. K-Nearest Neighbours(KNN)

An 80-20 train-test split was used which was followed by GridSearchCV to find the best hyperparameters for each model.

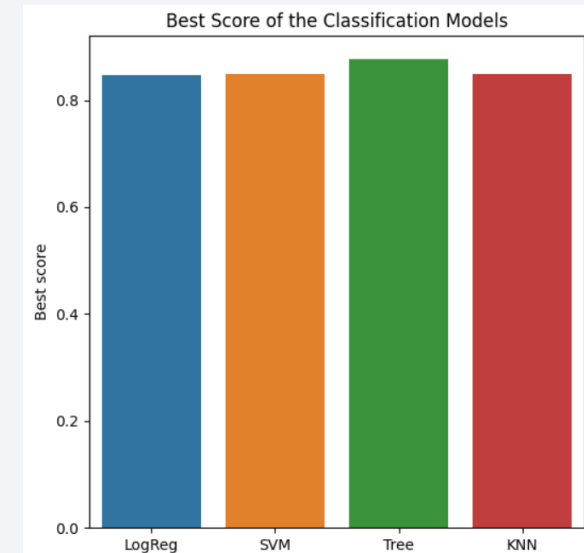
Results



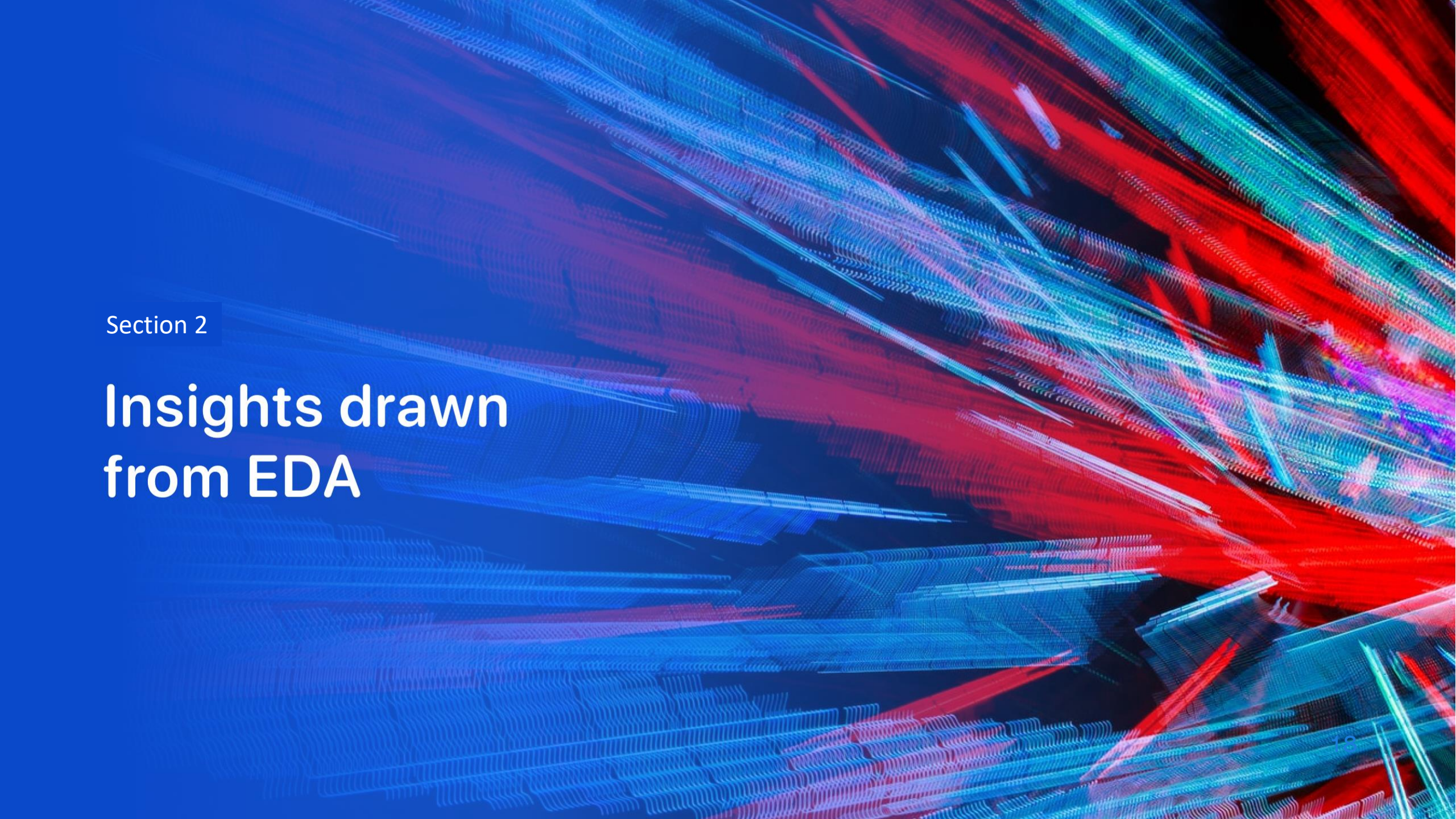
An increasing trend in success rate of Falcon-9 landings over the years.



Visualise the data in an interactive dashboard to observe patterns and trends.



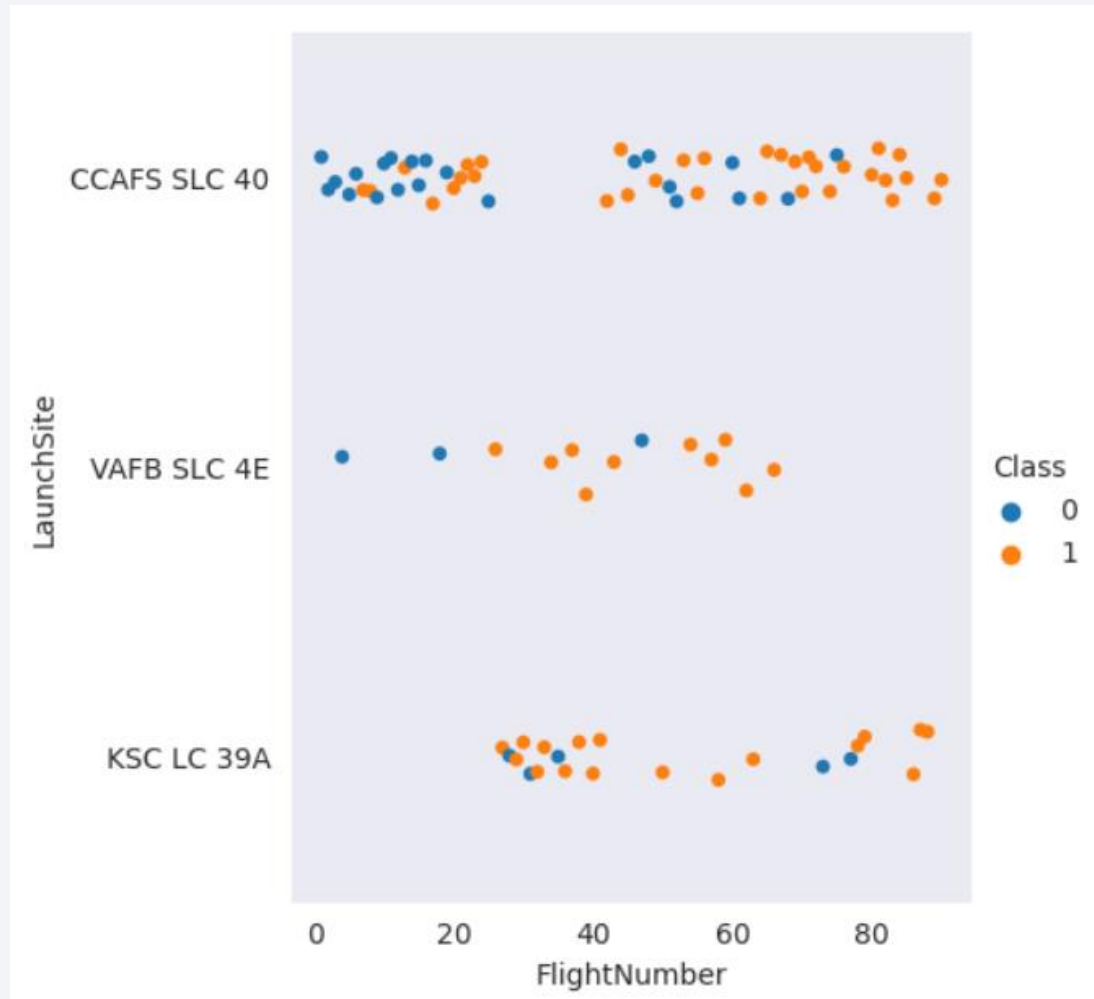
Choosing the best model based on its Best_Score that was derived from GridSearchCV.



Section 2

Insights drawn from EDA

Flight Number vs. Launch Site



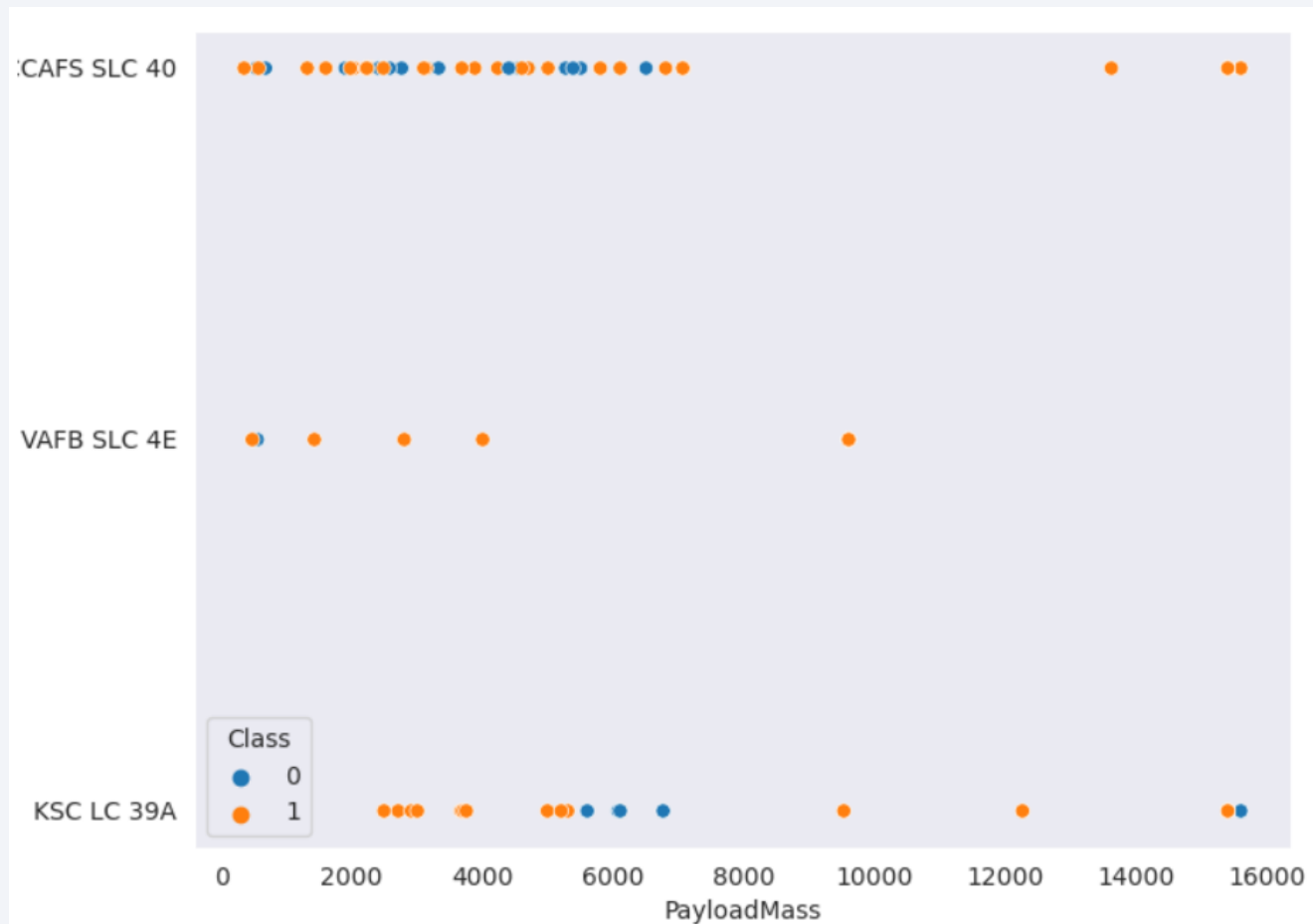
A class of 0 (blue) represents a landing failure while class of 1 (orange) represents a successful landing.

Based on the diagram, we can observe that the number of successful landings has been increasing in relation to the number of flights.

The vast majority of tests has been conducted at CCAFS SLC 40, with almost all early (<30) tests being held there.

KSC LC 39A was only used after flight 20, which could explain its higher success rate.

Payload vs. Launch Site

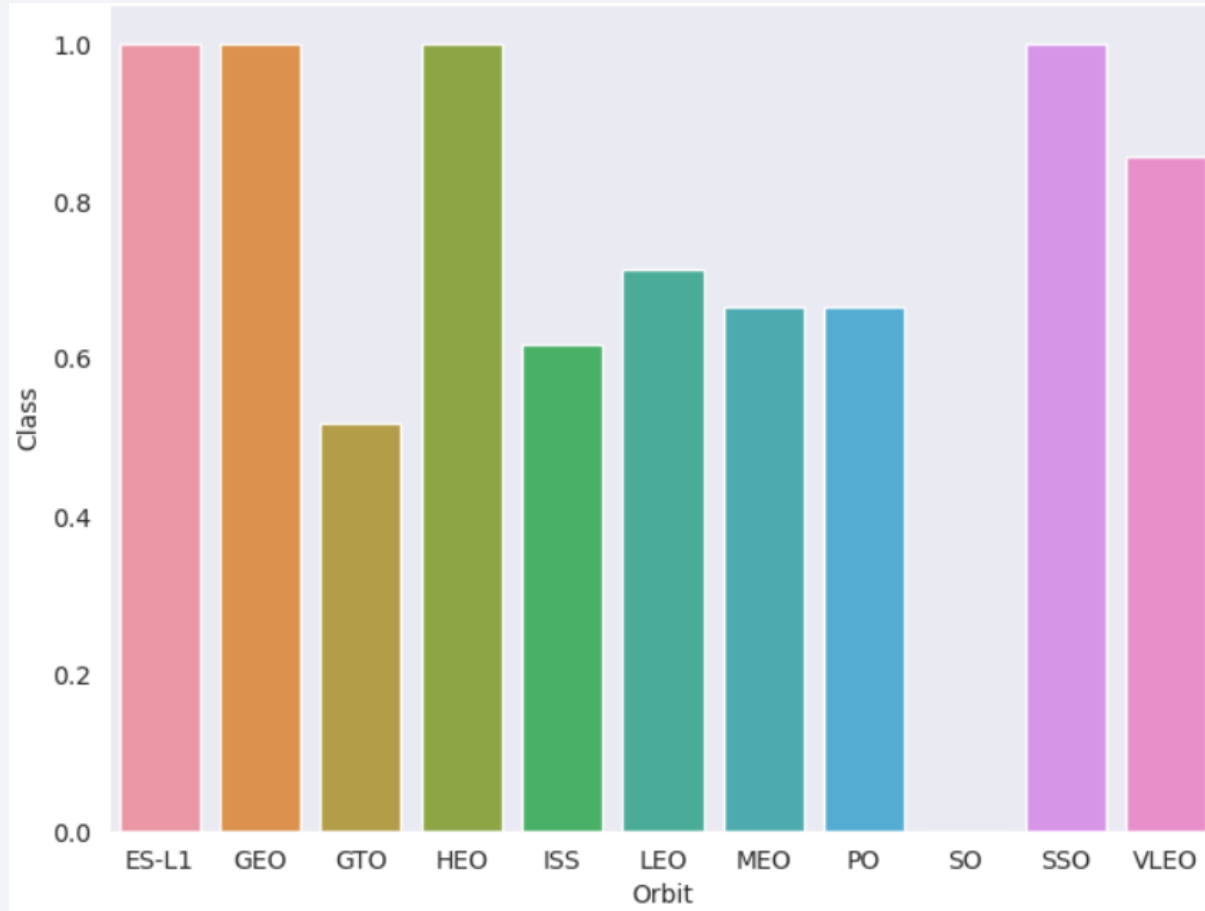


The majority of launches had a PayloadMass of less than 8000kg.

Although the points suggest that the success rate of heavier payloads is higher, this could be due a lack of data on heavier launches.

There does not appear to be any clear relationship between launch site, payload mass and a successful landing.

Success Rate vs. Orbit Type

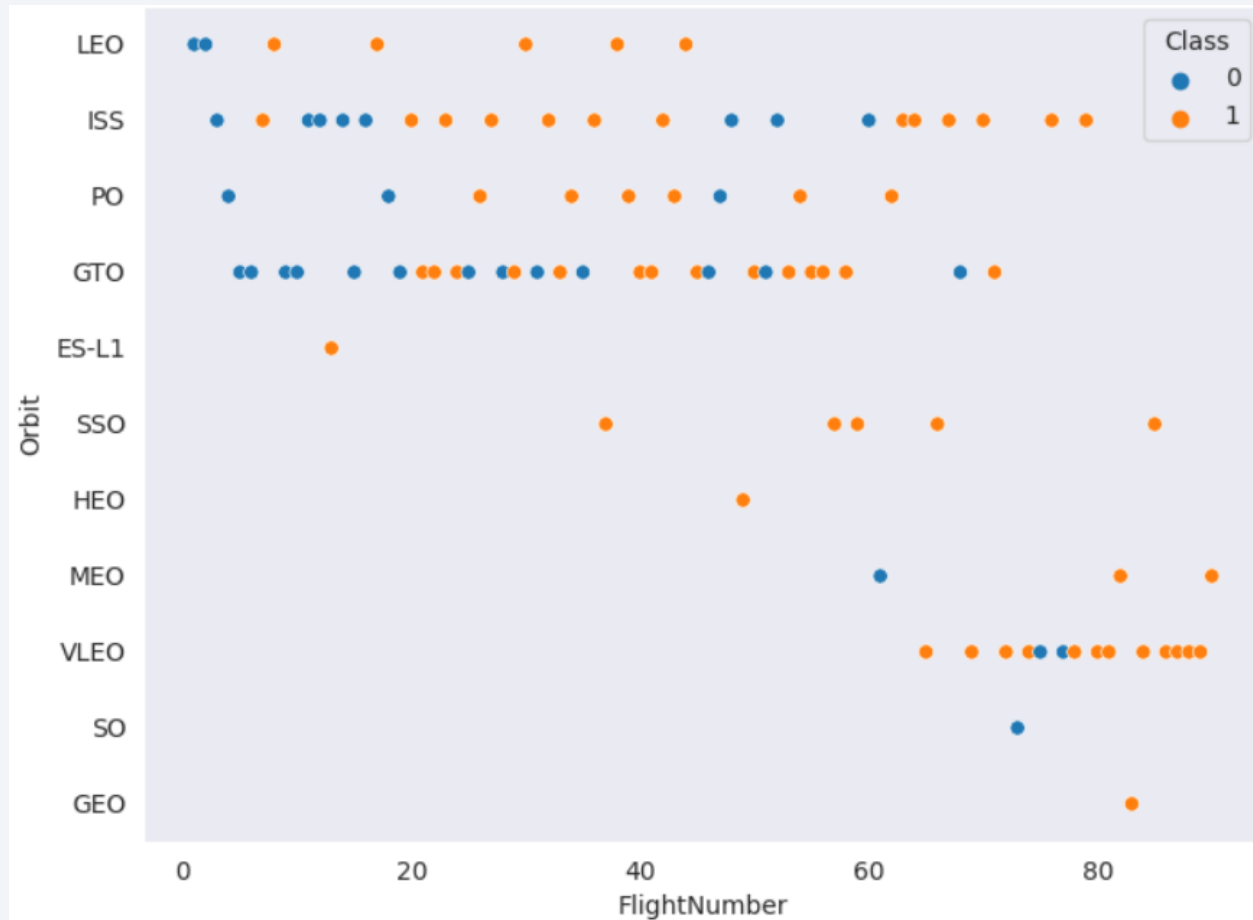


The success rate of 3 orbit types is 100%:

- ES-L1
- GEO
- HEO

At first glance, the orbit of SSO also has a success rate of 100% and SO has a success rate of 0%, however SSO and SO are the same orbit. Hence, the success rate is not obvious from this diagram.

Flight Number vs. Orbit Type



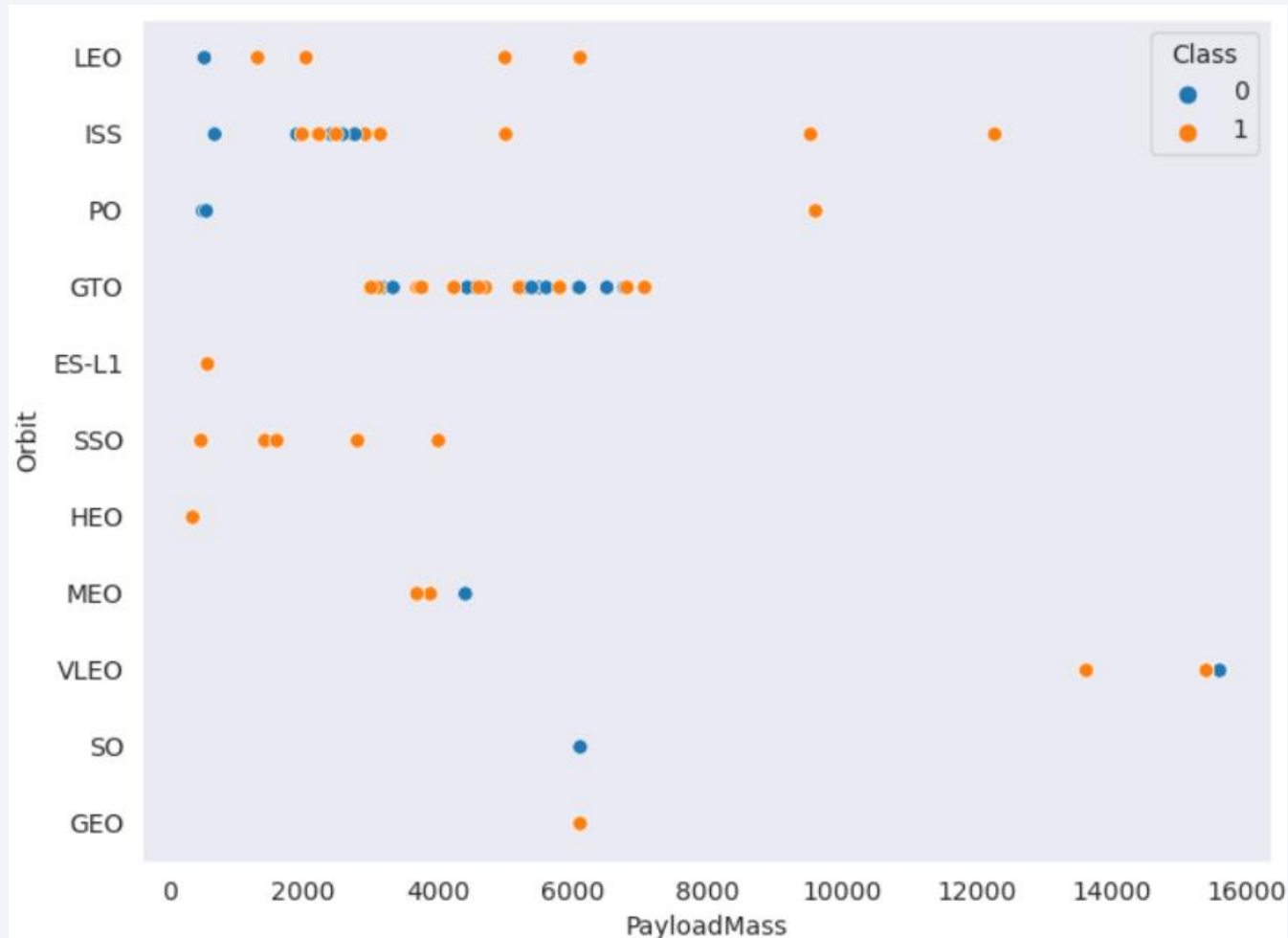
There does not appear to be any relationship between flight number and the success/failure of a landing from a specific orbit type.

In general, the success of landings is increasing as more flights are launched.

The high success rate of ES-L1, HEO and GEO can be attributed to each of them having only 1 launch.

For SSO/SO, we can see a total of 6 launches with 1 failure, thus its success rate is 83.3%.

Payload vs. Orbit Type



As the PayloadMass increases, these orbit types display a higher success rate:

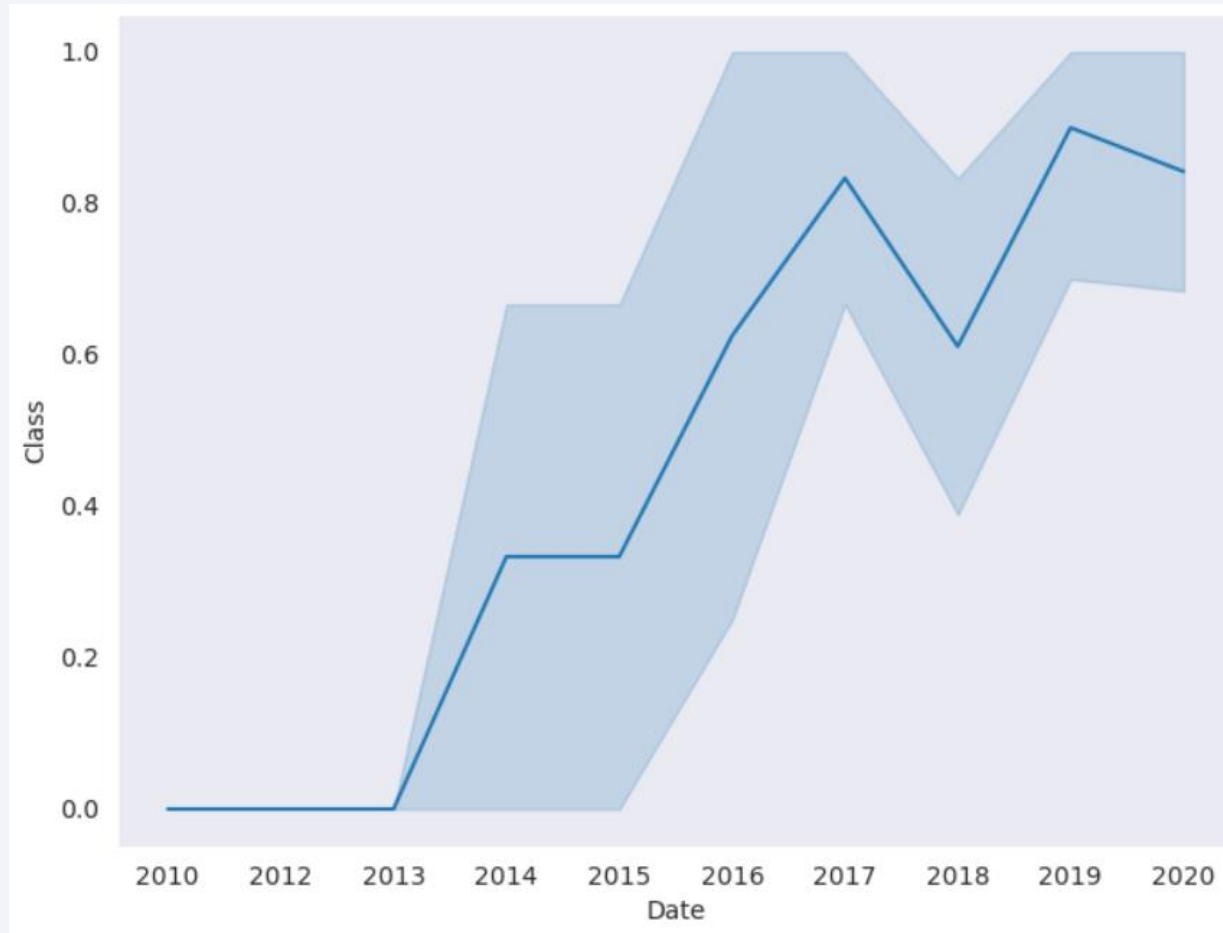
- ISS
- VLEO
- LEO

Conversely, these orbit types display a higher success rate with lesser payload:

- SSO
- HEO
- ES-L1

GTO does not appear to show any clear relationship between PayloadMass and success rate.

Launch Success Yearly Trend



From 2010-2013, there were no successful landings.

From 2013 onwards, there is a general increase in success of landings of the Falcon-9.

There was a dip in success rate from 2017 to 2018, but it appears to be temporary as it increased again the following year.

The latest data indicates a success rate greater than 80%.

All Launch Site Names

Find the names of the unique launch sites

Query:

```
%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTBL
```

Result:

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

The function DISTINCT in SQL only returns the unique values.

“Launch_Site” FROM SPACEXTBL determines the data from which the unique values are selected.

Launch Site Names Begin with 'CCA'

Find 5 records where launch sites begin with `CCA`

Query:

```
%sql SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE "CCA%" LIMIT 5
```

LIMIT 5 reduces the result to 5 rows.

LIKE "CCA%" returns strings that start with 'CCA'.

Result:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
04-06-2010	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
08-12-2010	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
22-05-2012	07:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
08-10-2012	00:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
01-03-2013	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

Calculate the total payload carried by boosters from NASA

Query:

```
%sql SELECT SUM(PAYLOAD_MASS__KG_) FROM SPACEXTBL WHERE "Customer" LIKE "NASA (CRS)"
```

Result:

SUM(PAYLOAD_MASS__KG_)

45596

The SUM function adds up the total of PAYLOAD_MASS__KG_.

The WHERE clause limits data to rows in which Customer is NASA (CRS).

Average Payload Mass by F9 v1.1

Calculate the average payload mass carried by booster version F9 v1.1

Query:

```
%sql SELECT AVG(PAYLOAD_MASS__KG_) FROM SPACEXTBL WHERE "Booster_Version" LIKE "F9 v1.1"
```

Result:

AVG(PAYLOAD_MASS__KG_)

2928.4

The AVG function returns the average of all the values in PAYLOAD_MASS__KG_.

The WHERE clause limits the data to rows in which the Booster_Version is F9 v1.1

First Successful Ground Landing Date

Find the dates of the first successful landing outcome on ground pad

Query:

```
%%sql SELECT min(substr(Date,7,4) || '-' || substr(Date,4,2) || '-' || substr(Date,1,2)) AS first_successful_landing_date
FROM SPACEXTBL WHERE "Landing _Outcome" = 'Success (ground pad)'
```

Result:

first_successful_landing_date

2015-12-22

The MIN function selects the earliest date from the Date column.

The WHERE clause limits the data to rows in which Landing _Outcome is “Success (ground pad)”.

Successful Drone Ship Landing with Payload between 4000 and 6000

List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

Query:

```
%%sql SELECT Booster_Version FROM SPACEXTBL WHERE ("Landing _Outcome" = 'Success (drone ship)')  
AND ("PAYLOAD_MASS__KG_" BETWEEN 4000 AND 6000)
```

Result:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

The WHERE clause filters the data to the rows in which Landing _Outcome is Success (drone ship).

The AND function allows the user to add additional conditions to the query that must also be satisfied.

BETWEEN 4000 and 6000 limits the range of PAYLOAD_MASS__KG_ such that it is greater than 4000 and less than 6000.

Total Number of Successful and Failure Mission Outcomes

Calculate the total number of successful and failure mission outcomes

Query:

```
%sql SELECT Mission_Outcome, COUNT(Mission_Outcome) AS Total_Number FROM SPACEXTBL GROUP BY Mission_Outcome
```

Result:

Mission_Outcome	Total_Number
Failure (in flight)	1
Success	99
Success (payload status unclear)	1

The COUNT function adds up the total number of a repeated value in the Mission_Outcome column and returns in a new column labelled Total_Number.

The GROUP BY function is used to group the results based on the type of Mission_Outcome.

Boosters Carried Maximum Payload

List the names of the booster which have carried the maximum payload mass

Query:

```
%%sql SELECT DISTINCT(Booster_Version) FROM SPACEXTBL
WHERE PAYLOAD_MASS__KG_ = ( SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL)
```

In the subquery, the MAX function returns the maximum value of PAYLOAD_MASS__KG_.

This value is then used by the main query in the WHERE clause to determine which Booster_Version has carried the maximum payload mass.

Result:

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

Query:

```
%%sql SELECT substr(Date, 4, 2) AS "Month", "Landing _Outcome", Booster_Version, Launch_Site FROM SPACEXTBL
WHERE (substr(Date,7,4) = '2015') AND "Landing _Outcome" = "Failure (drone ship)"
```

Result:

Month	Landing _Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

The WHERE clause limits the data to rows in which the year is 2015.

The AND adds a second condition which is that the Landing _Outcome is Failure (drone ship).

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Rank the count of successful landing_outcomes between the date 04-06-2010 and 20-03-2017 in descending order.

Query:

```
%%sql SELECT "Landing _Outcome", COUNT("Landing _Outcome") AS Total_Number, "Date" FROM SPACEXTBL
WHERE substr(Date,7,4) || substr(Date,4,2) || substr(Date,1,2) BETWEEN '20100604' AND '20170320'
AND "Landing _Outcome" LIKE "Success%"
GROUP BY "Landing _Outcome"
ORDER BY COUNT("Landing _Outcome") DESC
```

Result:

Landing _Outcome	Total_Number	Date
Success (drone ship)	5	08-04-2016
Success (ground pad)	3	22-12-2015

The WHERE limits the data to between 04-06-2010 and 20-03-2017 when used with the BETWEEN and AND.

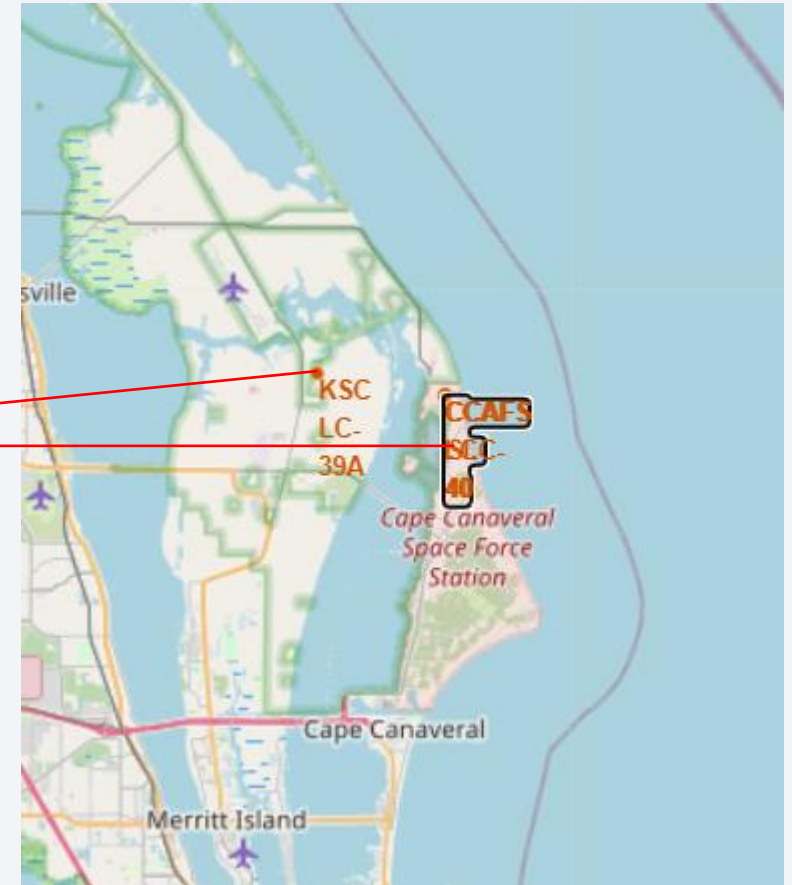
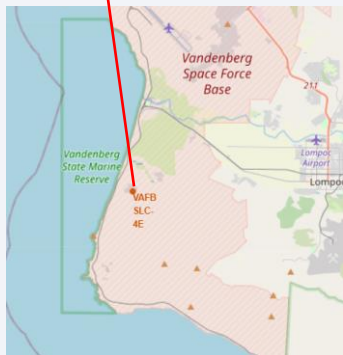
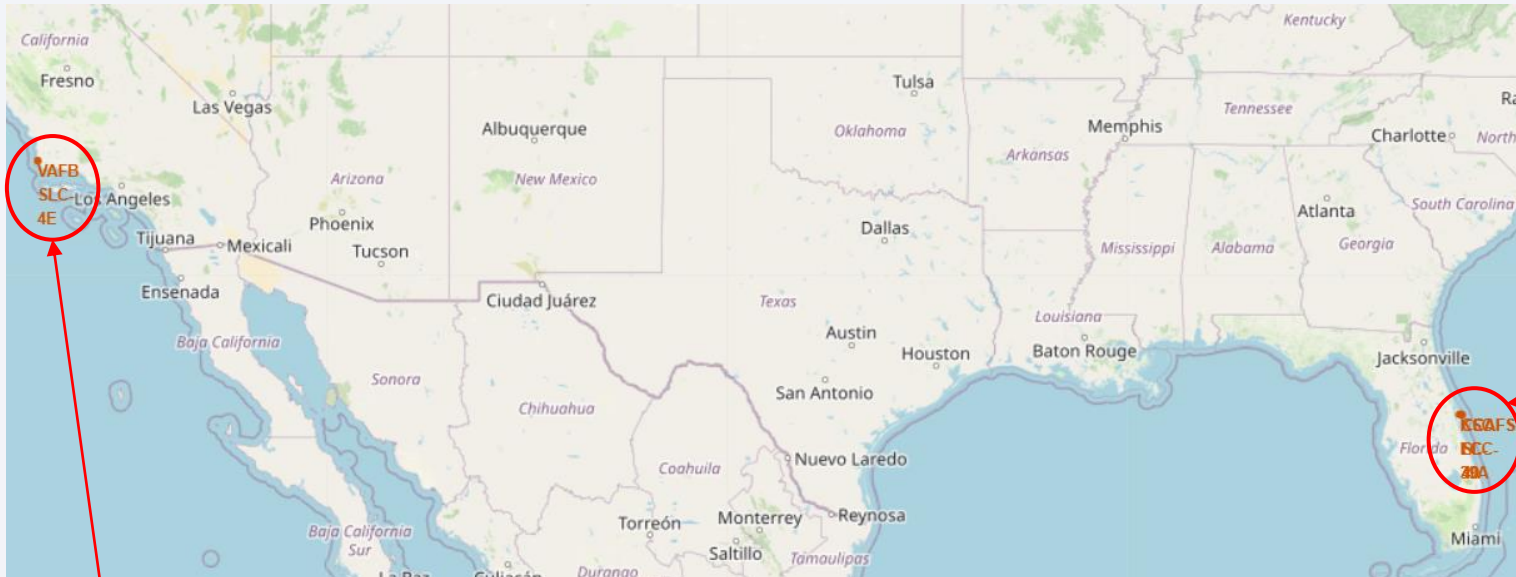
The GROUP BY and ORDER BY group and order the data based on Landing _Outcome in descending order due to DESC.

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

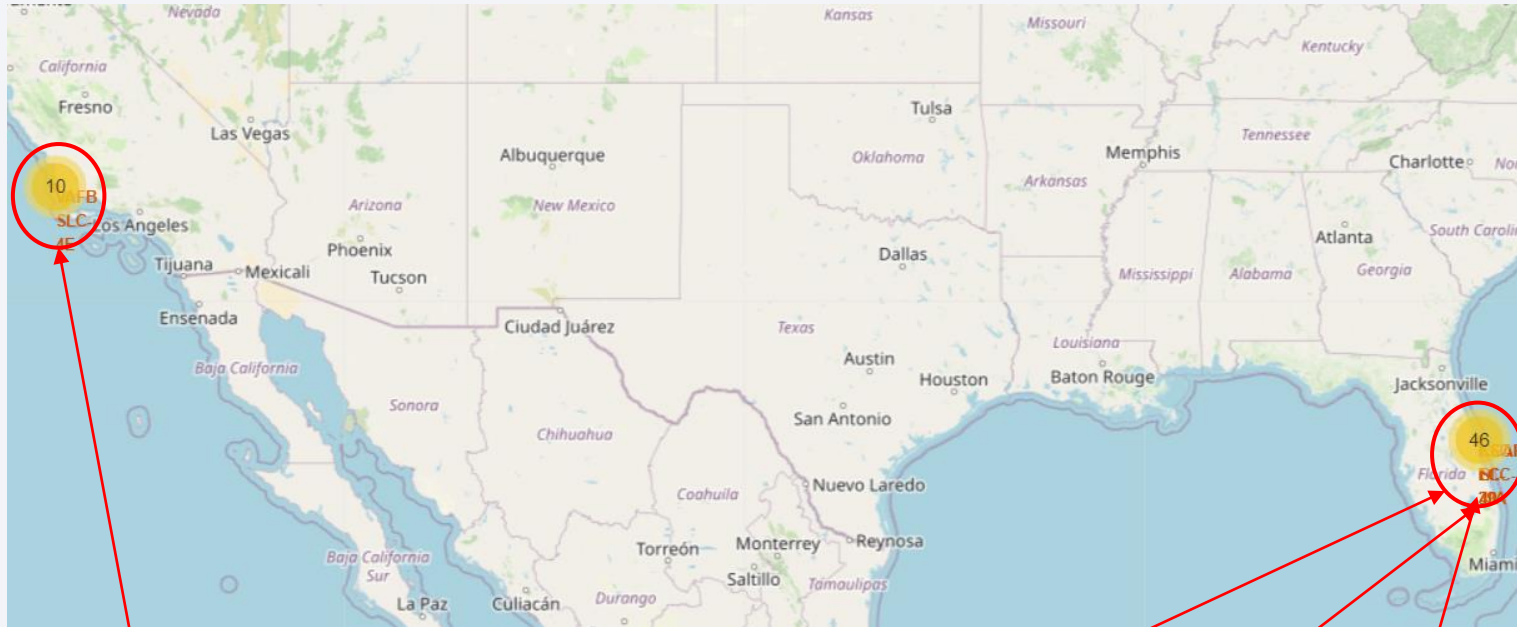
Launch Sites Proximities Analysis

All Launch Sites on a Map



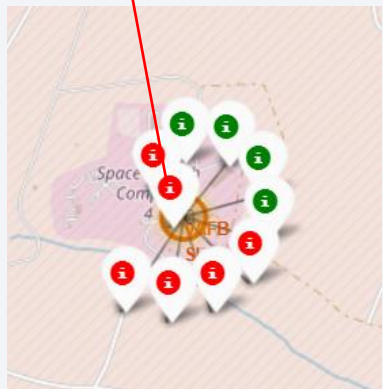
All the launch sites are near the coasts of USA. They are also close to the equator as the sites are all located close to the southern border of USA.

Colour-Labelled Launch Sites for Success/Failure

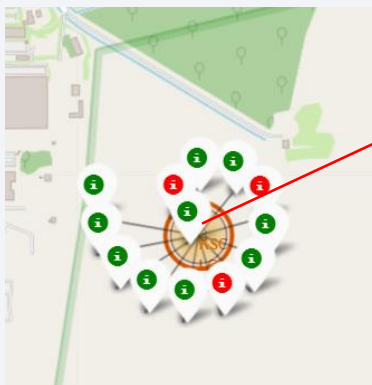


The markers are now labelled with the number of launches that took place at that launch site.

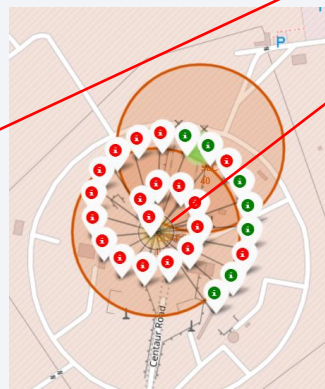
When clicked on, they will show a spiral on markers. In this spiral, the **green** markers indicate a successful landing while the **red** ones indicate a failed landing.



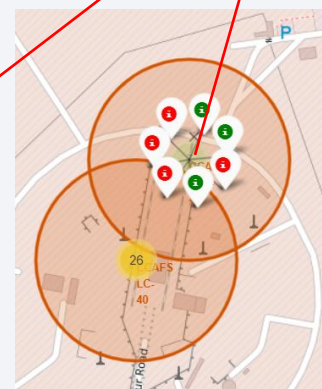
VAFB SLC-4E



KSC LC-39A

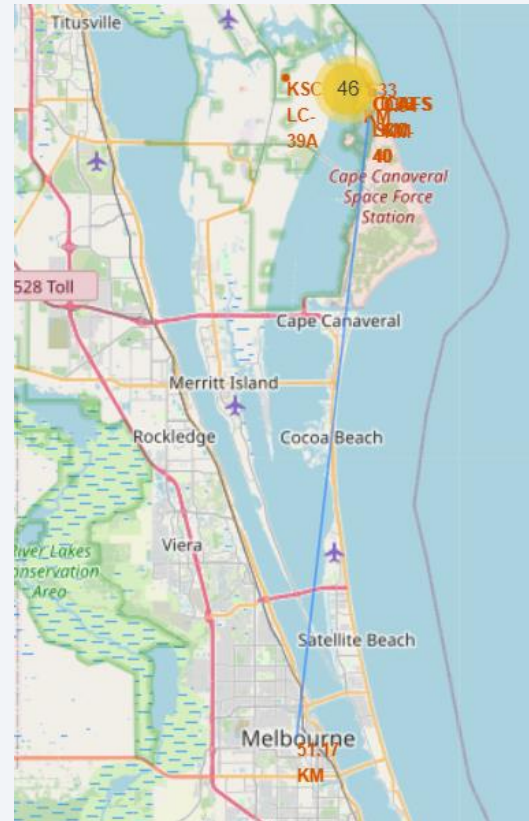
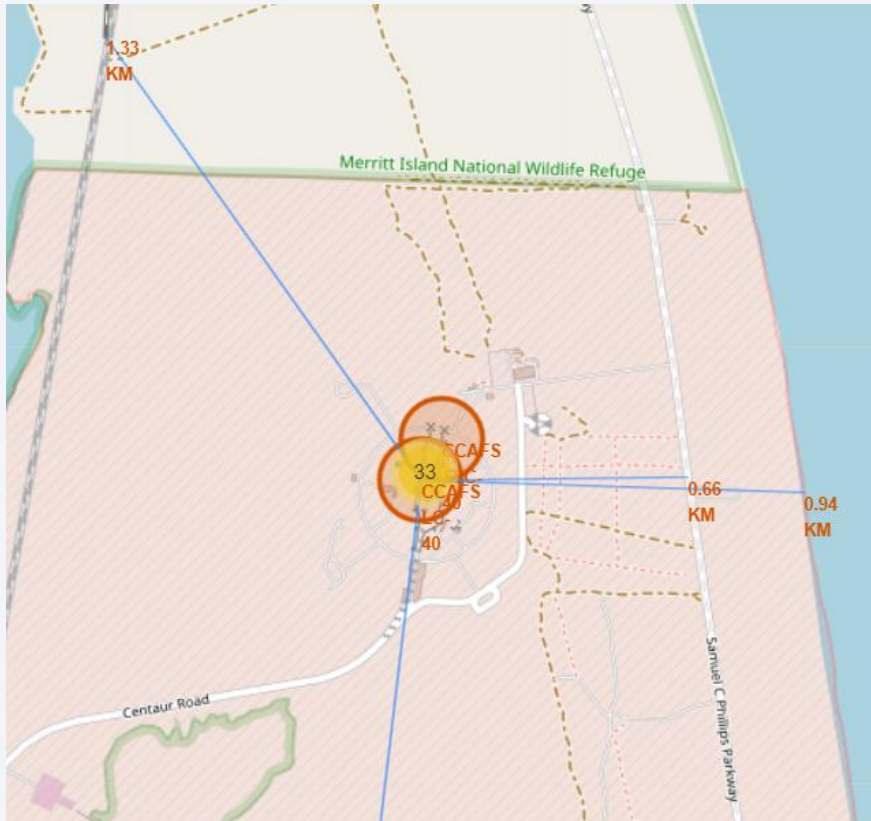


CCAFS LC-40



CCAFS SLC-40

Proximities of Launch Site in Florida

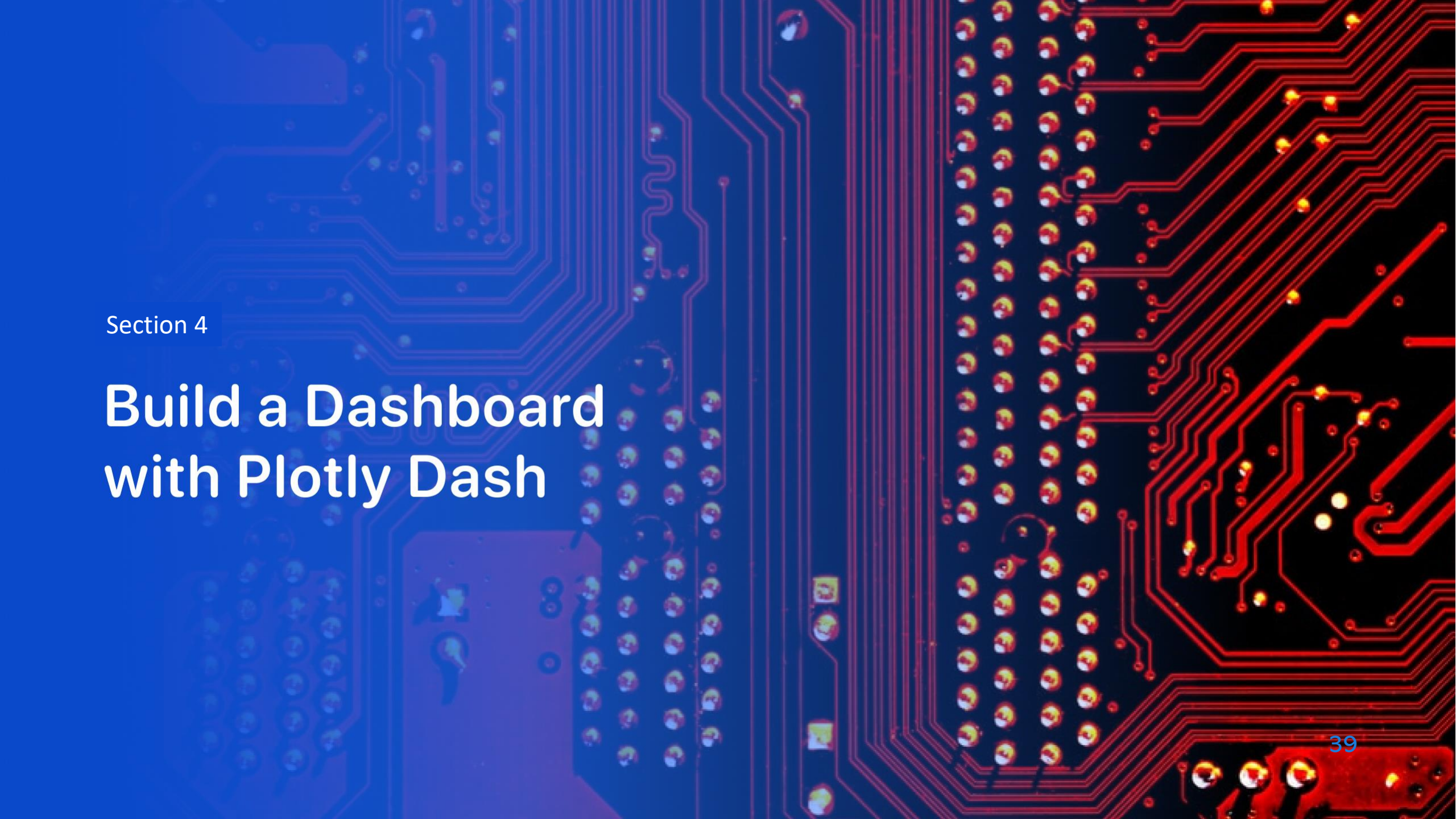


Do launch sites keep certain distance away from cities? Yes, around 51km.

These distances are likely to ensure that the site is well supplied, yet far enough away from major population centers to prevent any accidents from occurring.

Its proximity to the coast allows any mishaps to crash safely into the ocean instead on endangering people on land.

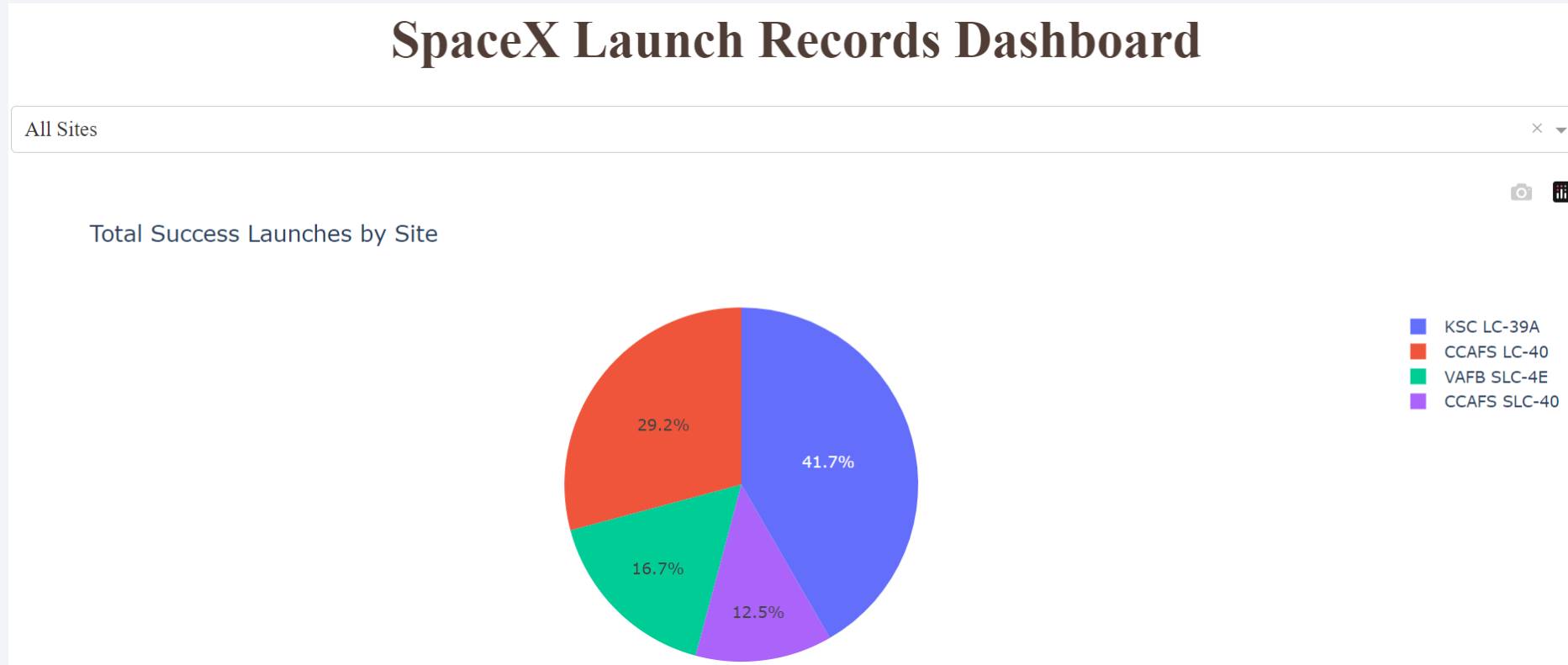
Are launch sites in close proximity to railways? Yes, around 1.33km.
Are launch sites in close proximity to highways? Yes, around 0.66km.
Are launch sites in close proximity to coastline? Yes, around 0.94km.



Section 4

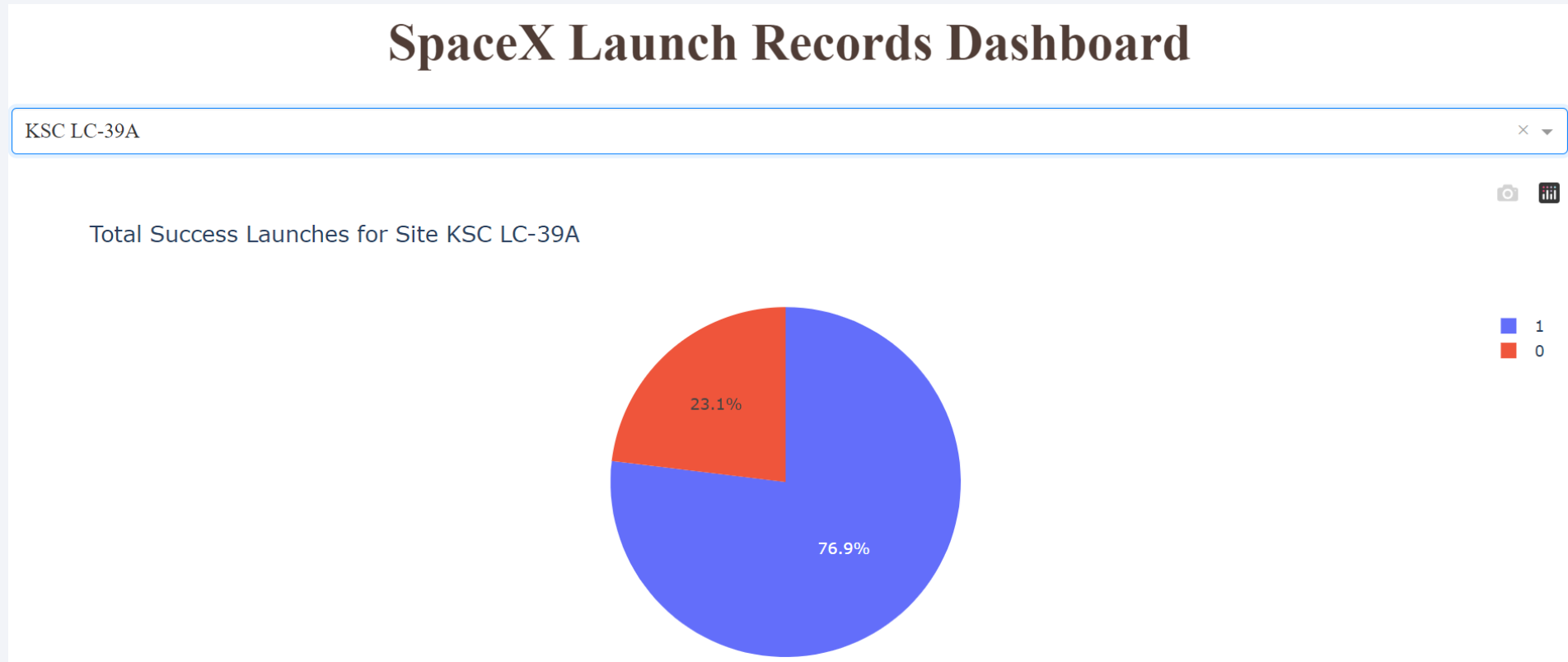
Build a Dashboard with Plotly Dash

Total Successful Launches by Site



The pie chart allows us to compare the percentage of overall successful launches that took place at each site. The most successful launch site is KSC LC-39A with 41.7% of all successful launches.

Launch Site with Highest Launch Success Ratio



The pie chart shows the success and failure rate of KSC LC-39A. It has the highest launch success rate of 76.9%.

Payload vs. Launch Outcome scatter plot for all sites



- The scatter plot above shows all the data points. The 2 scatter plots on the right are split into light and heavy payload weights:
1. Light (0-5000KG)
 2. Heavy (5000-10000KG)

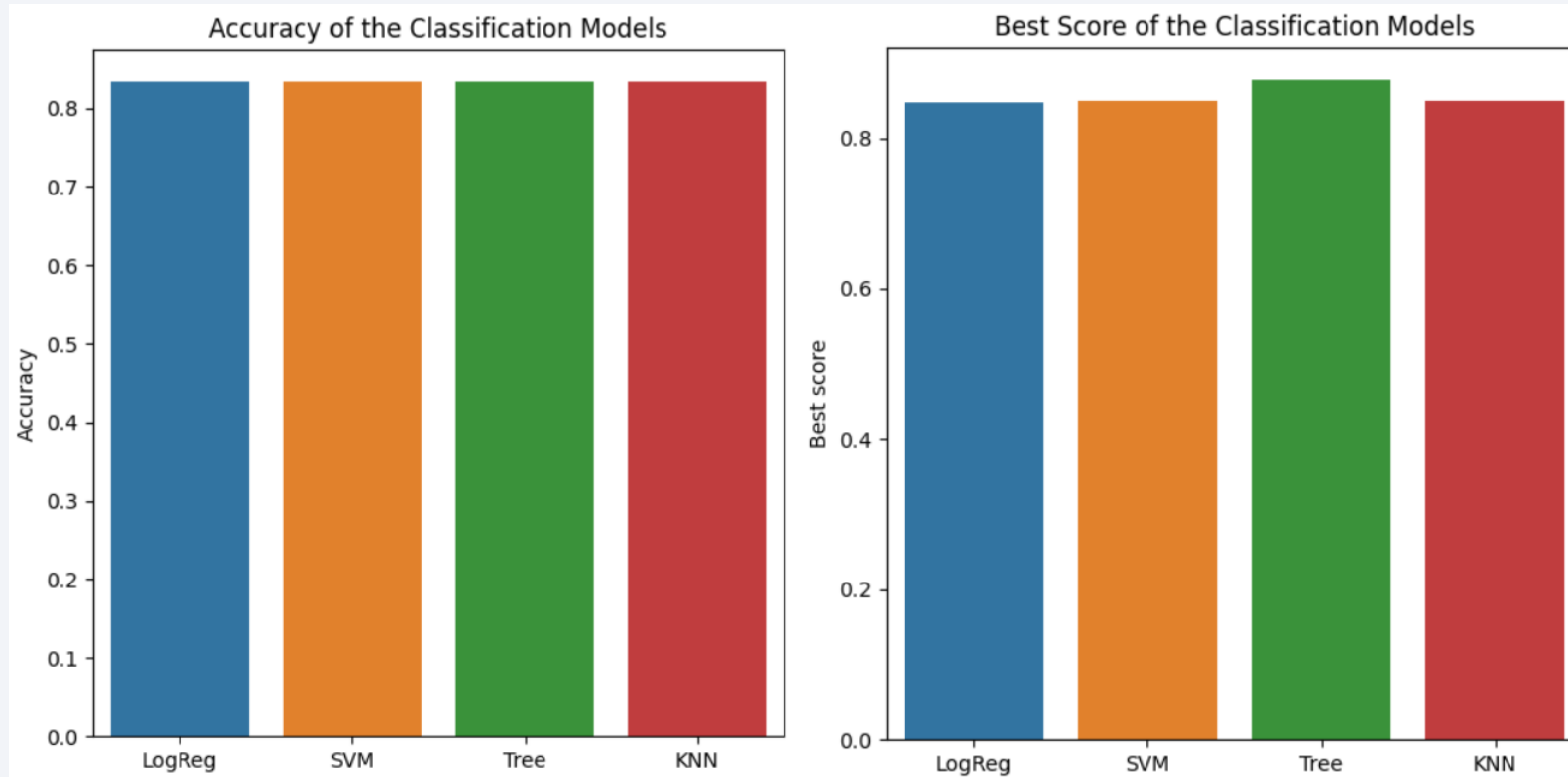
We can observe that the success rate (Class=1) for launches with a lighter payload is much higher than those with heavier payloads.



Section 5

Predictive Analysis (Classification)

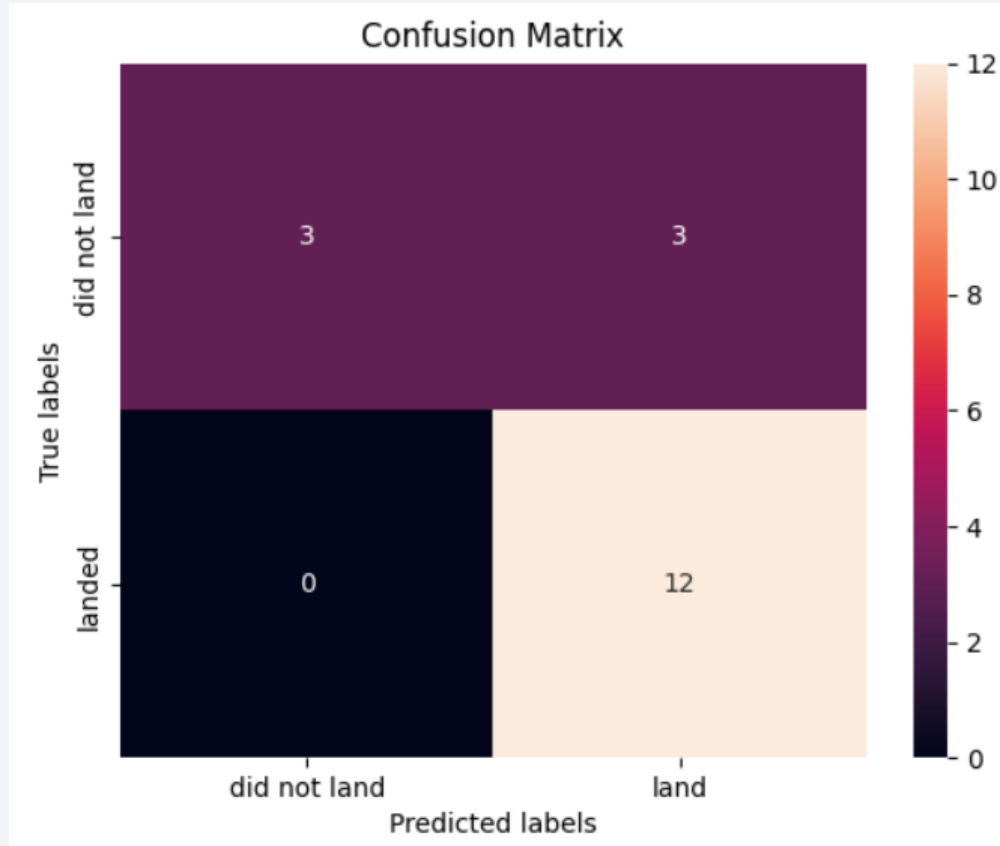
Classification Accuracy



	ML_Model	Accuracy	Best_Score
0	LogReg	0.833333	0.846429
1	SVM	0.833333	0.848214
2	Tree	0.833333	0.876786
3	KNN	0.833333	0.848214

We can see that the Accuracy of all the models on the test set is exactly the same. However, based on the best hyperparameters obtained from GridSearchCV, the Decision Tree is the best model to use, with a score of 87.7%.

Confusion Matrix



The Confusion Matrix for all the models are exactly the same as they all had the same accuracy.

The models were all able to predict 15 out of 18 labels correctly. (12 True Positives and 3 True Negatives). The model incorrectly predicted 3 landings as failure when they were actually successful landings. (False Positives)

Conclusions

- The main trend observed is that the success rate of Falcon-9 landings has been increasing over the years.
- The success rate varies based on Orbit type, with ES-L, GEO, HEO having a 100% success rate.
 - Their success could be attributed to them having only 1 flight each.
 - Thus SSO/SO is more impressive as it has a 83% success rate with 6 total launches.
- The launch site with the most number of successful launches is KSC LC-39A at 41.7%. It also has the highest individual success rate at 76.9%.
- Some interesting features of these launch sites are that they are usually located in the proximity of the coast, highways and railways and geographically near the equator.
- Falcon-9 landings with heavier payloads have a lower success rate than lighter payloads.
- The best model overall will be the Decision Tree model as it has the highest Best_Score of 87.7% and has the same Accuracy as the other models at 83.3%.

Appendix – GitHub URL

Web Scrapping Code

```
extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable plainrowheaders collapsible")):
    # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to Launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
        #get table element
        row=rows.find_all('td')
        #if it is number save cells in a dictionary
        if flag:
            extracted_row += 1
            # Flight Number value

            launch_dict["Flight No."].append(flight_number)

            datatimelist=date_time(row[0])
            # Date value

            date = datatimelist[0].strip(',')
            launch_dict["Date"].append(date)

            # Time value

            time = datatimelist[1]
            launch_dict["Time"].append(time)

            # Booster version

            bv=booster_version(row[1])
            if not(bv):
                bv=row[1].a.string
            launch_dict["Version Booster"].append(bv)
```

```
# Launch Site

launch_site = row[2].a.string
launch_dict["Launch site"].append(launch_site)

# Payload

payload = row[3].a.string
launch_dict["Payload"].append(payload)

# Payload Mass

payload_mass = get_mass(row[4])
launch_dict["Payload mass"].append(payload_mass)

# Orbit

orbit = row[5].a.string
launch_dict["Orbit"].append(orbit)

# Customer

customer = row[6].text.strip()
launch_dict["Customer"].append(customer)

# Launch outcome

launch_outcome = list(row[7].strings)[0]
launch_dict["Launch outcome"].append(launch_outcome)

# Booster Landing

booster_landing = landing_status(row[8])
launch_dict["Booster landing"].append(booster_landing)
```

Thank you!

